

INF-239 Bases de Datos

Tarea 2: PHP + SQL

Profesores: Ricardo Salas (ricardo.salas@usm.cl)

Mauricio Figueroa (mauricio.figueroac@usm.cl)

Ayudante Coordinador: Gonzalo Alarcón (gonzalo.alarconc@usm.cl)

Marzo, 2026

1. Objetivo

Investigar y aplicar conceptos básicos de creación de aplicaciones web con base de datos integrada, utilizando PHP y MySQL, incorporando además la normalización a 3FN, aplicada al caso de digitalización del formulario de postulación de iniciativas del CT-USM.

2. Especificaciones y reglas

El desarrollo de esta tarea debe cumplir las siguientes especificaciones, de lo contrario existirá un descuento en la nota final:

- Se debe desarrollar una aplicación web básica en parejas, utilizando PHP como herramienta de desarrollo y MySQL para gestionar la base de datos.
- La tarea debe ser entregada el **jueves 07 de Mayo hasta las 23:59 horas** vía AULA.
- Se dará un **bonus de 5 puntos** a quienes desarrollen frontend (diseño de la página), por ejemplo usando Bootstrap. No habrá descuento para quienes no desarrollen frontend, pero la información que se debe mostrar en la página debe verse de manera clara y no desordenada; en caso de desorden sí habrá descuento. Los puntos solo se darán a alumnos cuya nota sea menor a 100, siendo el tope nota 100. Los puntos no son acumulables ni transferibles a otras evaluaciones, solo válidos en esta tarea.
- Se dará un **bonus de 5 puntos** a quienes implementen en el sistema protección a inyecciones SQL. Los puntos solo se darán a alumnos cuya nota sea menor a 100, siendo el tope nota 100. Los puntos no son acumulables ni transferibles a otras evaluaciones, solo válidos en esta tarea.

Sobre la entrega

La tarea debe ser entregada como un archivo comprimido de la forma **T2-ROL1-ROL2.zip** y debe contener los siguientes archivos:

1. **Carpeta PDF:** Un archivo **.pdf** el cual contenga los pasos para la Normalización 3FN.
2. **Carpeta PHP:** Todos los archivos **.php** deben estar contenidos aquí. Puede crear los archivos que sean necesarios para resolver su tarea.
3. Los archivos **.js** o **.css** deben estar ordenados dentro de carpetas que contengan archivos del mismo estilo.
4. Se debe incluir un archivo **README** (**.txt** o **.md**), el cual debe contener nombres, rol de los alumnos y las instrucciones para la correcta ejecución del programa. Ante cualquier duda sobre algo que no aparezca especificado en la tarea y no afecte las reglas de esta, puede asumir lo que estime conveniente, pero debe ser especificado en el README. En caso de asumir situaciones y no especificarlas, habrá descuento.

5. Esta tarea debe ser defendida para obtener la nota. Para esto se habilitará un documento donde podrán inscribir cuándo realizar su defensa según los horarios disponibles del ayudante que les revisará. Quienes inscriban un horario y no asistan tendrán 0 como nota de esta entrega.
6. Es responsabilidad del equipo estar a la hora seleccionada en el lugar indicado.
7. **Las consultas se deberán hacer mediante el foro de Aula, el cual se encontrará en la sección Tareas.**

Reglas

- **Debe** realizarse en parejas, no se aceptarán tareas individuales.
- Las copias serán evaluadas con nota 0 y se informará a las respectivas autoridades. Consultas vía la plataforma oficial AULA.
- En el foro de búsqueda de pareja podrán buscar pareja quienes no tengan; esto es exclusiva responsabilidad del estudiante.
- En caso de problemas con su pareja podrán contactar al profesor explicando su situación. Solo un alumno debe realizar la entrega.
- En caso de que falle la ejecución de algún comando, no se asignará puntaje a éste. Existe la posibilidad de que a su defensa asista su profesor y realice preguntas.
- Es responsabilidad del estudiante inscribir un horario de defensa y estar presente en la fecha y hora elegida.
- Cada grupo tendrá un horario definido para su defensa; en caso de atraso contarán con un tiempo menor para presentar su trabajo.
- La información respecto a la defensa será eventualmente publicada en Aula, esto considera el detalle sobre los descuentos. Es obligación del estudiante estar atento a esta información y cumplir con lo establecido allí.
- Las consultas se responderán hasta 72 horas antes de la entrega.

3. Descripción del problema

El Centro Tecnológico de la USM (CT-USM) ha solicitado digitalizar el procedimiento de postulación de iniciativas para evaluación y financiamiento, utilizando como base el formulario de postulación y sus consideraciones (tipo de iniciativa, regiones de ejecución/impacto, empresa externa con tamaño y convenio, sede/campus, equipo mínimo y cronograma).

En la Tarea 1 se modeló e implementó la base de datos para este caso (modelo conceptual, lógico/relacional, físico en MySQL, poblamiento y consultas). Para esta Tarea 2 se solicita implementar una aplicación web en PHP + MySQL que permita operar el proceso de postulación mediante funcionalidades coherentes con el formulario, integrando una base de datos normalizada.

Para ayudarles a organizar todo el trabajo por hacer, se les deja la siguiente lista de actividades:

3.1. Normalización

- Deben terminar/corregir el modelo creado a partir de la Tarea 1.
- Deben convertir el modelo a 3FN mostrando todos los pasos.
- Los pasos pueden ser desarrollados a mano o de forma digital, pero deben estar ordenados.
- Se debe basar en la vista de usuario ya entregada en la Tarea 1, que se anexa nuevamente en este documento.

3.2. General

- Debe utilizar la base de datos normalizada. Dispone de la herramienta diseñador en phpMyAdmin (MySQL) (**no está permitido utilizar otra herramienta**) o puede generarla con sentencias. No olvide las claves primarias, foráneas y eliminación en cascada, entre los otros conceptos aprendidos para el modelo relacional.
- Se debe hacer uso de al menos un **Procedimiento Almacenado**, el cual debe ser funcional y acorde al contexto de la tarea.
- Se debe hacer uso de al menos un **Trigger**, el cual debe ser funcional y acorde al contexto de la tarea.
- El sistema debe tener **navegación entre secciones** (recargar la página o retroceder con el navegador no es válido).
- Se debe hacer uso de al menos una **View**, la cual debe ser funcional y acorde al contexto de la tarea.
- Se debe hacer uso de al menos una **Function SQL**, la cual debe ser funcional y acorde al contexto de la tarea.

3.3. Login

Crear login para entrar a la plataforma, que incluya los siguientes roles:

- **ROL 1 – Responsable académico (Postulante):** crea/edita postulaciones en estado “borrador” y las envía.
- **ROL 2 – Evaluador CT-USM (Coordinador de proyectos):** revisa postulaciones y registra evaluación/estado.
- **ROL 3 (Administrativo) – Administrador CT-USM:** gestiona evaluadores y asignaciones.

3.4. Página principal

1. **Barra de búsqueda:** permitir buscar postulaciones por un trozo de texto (por ejemplo: nombre de la iniciativa, empresa o código). Ambos roles pueden usarla.
2. **Listado general:** sección que permita visualizar postulaciones “gestionadas” (por ejemplo: enviadas/evaluadas), mostrando al menos: código/postulación N°, iniciativa, empresa, sede, región de ejecución/impacto, presupuesto y estado. Ambos roles pueden usarla, con visibilidad según rol.
3. **Mis postulaciones (Responsable académico):** sección que muestre todas las postulaciones donde el usuario sea responsable. Desde aquí se realizan acciones del caso de uso “Acceso y edición posterior al envío” (ver anexo).
4. **Crear postulación:** sección para ingresar lo necesario para registrar una postulación conforme al formulario CT-USM (incluye empresa externa, iniciativa, equipo y cronograma).

3.5. Búsqueda avanzada

Implementar filtros combinables, por ejemplo:

- Región de ejecución, región de impacto, sede/campus, tipo de iniciativa (nueva/existente), tamaño de empresa, convenio marco (Sí/No), estado de la postulación, evaluador asignado.

En los resultados debe aparecer un resumen claro: código, iniciativa, empresa, sede, regiones y estado.

3.6. CRUD Postulaciones (y subcomponentes)

Desarrollar CRUD para:

- Postulaciones (cabecera).
- Equipo de trabajo (integrantes).
- Cronograma (etapas).
- Empresa externa (al menos lectura y asociación; si implementan CRUD completo, mejor).

4. Aclaraciones

1. Cualquier supuesto debe ser especificado de forma explícita en el README; habrá un descuento por cada supuesto no especificado.
2. Function SQL, trigger, view y procedimiento almacenado deben ser incluidos en algunos de los puntos solicitados en la tarea. De aplicarse en algo no solicitado anteriormente, tendrán únicamente la mitad del puntaje.
3. Obligatoriamente deben existir suficientes datos de prueba para probar cada inciso de la tarea. Por ejemplo, si hipotéticamente se solicitase un top 10, no coloque exactamente 10 datos en su BD; coloque unos 12 para que el ayudante compruebe que efectivamente se realiza una selección correctamente.
4. Se recomienda iniciar con tiempo esta tarea.
5. **No se dará más plazo para esta entrega.**

5. Anexo (Vista Formulario Postulación)

[illegible]

Figura 1: Enter Caption

6. Referencias a material de apoyo

1. DSouza, D. (2022, January 17). *How to create a XAMPP MySQL Database in 9 easy steps*. Learn — Hevo. <https://hevo.com/learn/xampp-mysql/>

2. Web, C. M. [@carlosmasterweb]. (n.d.). *¿Como descargar e instalar XAMPP en Windows en 2025?* [Video]. Youtube. Retrieved January 14, 2026, from <https://www.youtube.com/watch?v=Ucm3vPu6448>
3. UskoKruM [@UskoKruM2010]. (n.d.). *Curso de PHP y MySQL desde 0* [Video]. Youtube. Retrieved January 16, 2026, from http://www.youtube.com/playlist?list=PL_wRgp7nihyYXMZ5bBP6v0Q31fRRJLiud